

PROJECT: BUILD A SECURITY MONITORING SYSTEM AWS

... By Pallavi Khadse

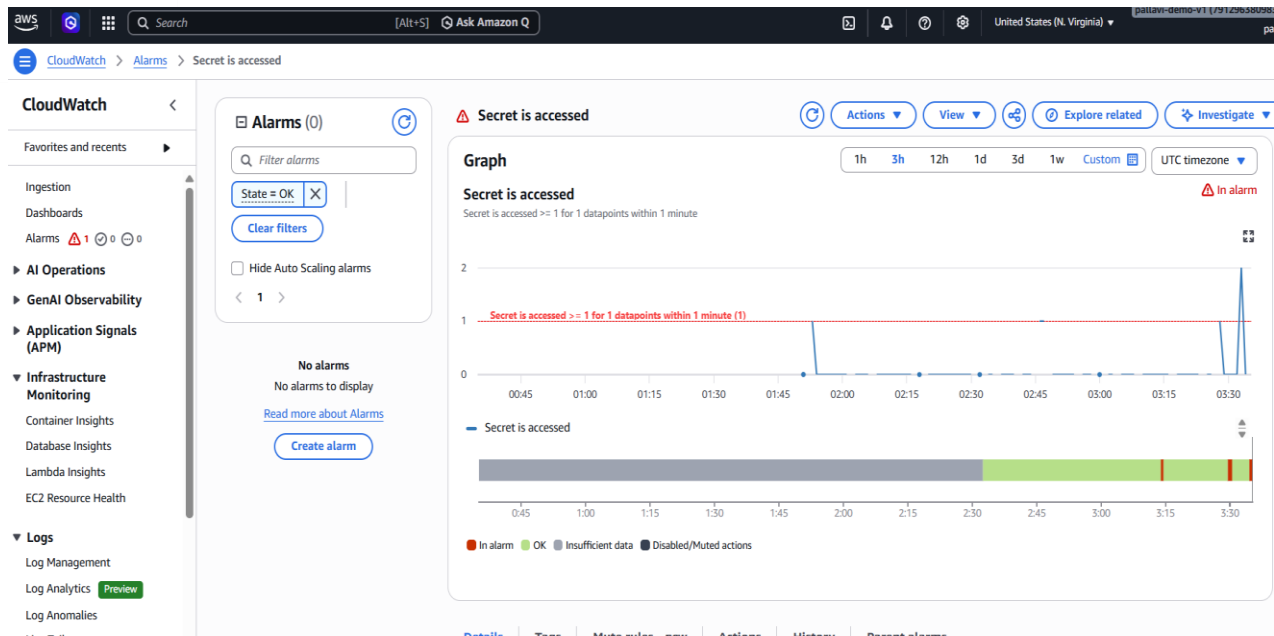


Fig. 1: CloudWatch alarm in its 'in alarm' state

Introducing Project!

In this project, I will demonstrate "a Security Monitoring System and SNS alerts in email" on AWS.

Following are the steps taken:

1. Set up AWS CloudTrail to track secret access events.
2. Use AWS CloudWatch to log access attempts and trigger notifications.
3. Create SNS alerts to get notified when your secrets are accessed.
4. Build a second notification system and compare which approach delivers better security alerts.

I'm doing this project to learn various AWS services such as IAM user, Secrets Manager, CloudTrail Monitoring, S3, Logging, CloudWatch Alarm, SNS, AWS CLI, etc.

Create a Secret

Secrets Manager is an AWS service to store, rotate, monitor, and control access to secrets such as database credentials, API keys, and OAuth tokens.

To set up for my project, I created a secret called "MySecretInfo" that contains Secret key-value pair.

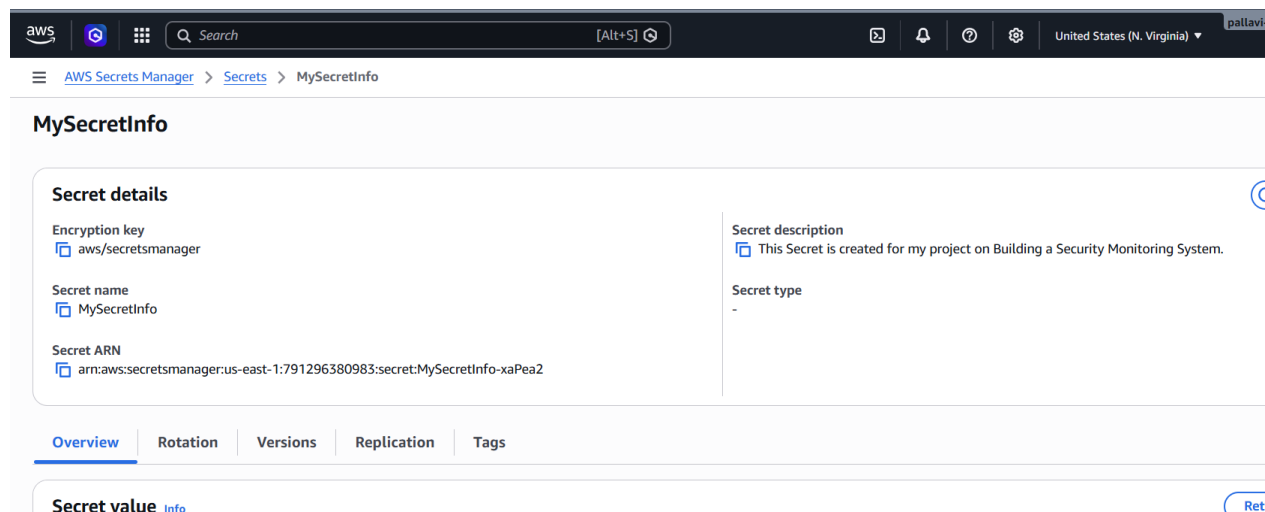


Fig.2: Secret's details page_AWS Secrets Manager

Set Up CloudTrail

CloudTrail is a monitoring service by AWS. I set up a trail to record events that happened in our AWS account, like creating resources, updating a name or setting, and accessing secrets in Secrets Manager.

AWS CloudTrail tracks Management Events (resource configuration), Data Events (resource-level actions), Insights Events (unusual API patterns), and Network activity events. AWS CloudTrail logs API activities as Read-only (fetching data/listing resources) or Read-write (creating, modifying, or deleting resources).

Read vs Write Activity

Read activities fetch information without changing resources, while Write activities create, modify, or delete resources.

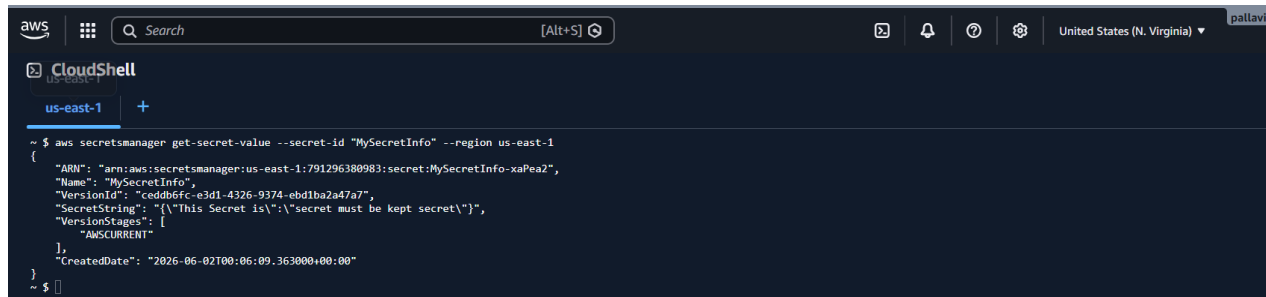
Read Activities: Operations that list, describe, or view data (e.g., DescribeInstances in EC2, ListBuckets in S3). They do not alter your infrastructure.

Write Activities: Operations that modify your environment (e.g., RunInstances in EC2, PutObject in S3, DeleteUser in IAM). They change your infrastructure or data state.

Verifying CloudTrail

I retrieved the secret in two ways:

1. through AWS Secrets Manager and,
2. using AWS CLI.



```

~ $ aws secretsmanager get-secret-value --secret-id "MySecretInfo" --region us-east-1
{
  "ARN": "arn:aws:secretsmanager:us-east-1:791296380983:secret:MySecretInfo-xaPeaz",
  "Name": "MySecretInfo",
  "VersionId": "ceddb6fc-e3d1-4326-937d-ebd1ba2a47a7",
  "SecretString": "{\"This Secret is\":\"secret must be kept secret\"}",
  "VersionStages": [
    "AMSCURRENT"
  ],
  "CreateDate": "2026-06-02T00:06:09.363000+00:00"
}
~ $

```

Fig. 3: Retrieving the secret using the AWS CLI

CloudWatch Metrics

CloudWatch Logs centralizes log data from AWS resources and applications to monitor, troubleshoot, and store log files.

It is important for Monitoring Centralized Storage: Collects logs from EC2, Lambda, CloudTrail, and applications in one place. Real-Time Monitoring: Tracks errors and metrics instantly using specific search patterns.

Automated Alerts: Triggers CloudWatch Alarms to notify teams when log errors spike.

Long-Term Retention: Stores historical log data securely for compliance and future analysis. Deep Analysis: Runs fast, SQL-like queries using CloudWatch Logs Insights to find issues.

CloudTrail stores historical records of who did what in your account for auditing, while CloudWatch stores operational data and application logs for real-time monitoring and alerting.

Key Differences:

Primary Purpose: CloudTrail is for security, compliance, and governance auditing. CloudWatch is for performance, application monitoring, and operational health.

Data Source: CloudTrail only records AWS API activity and user actions. CloudWatch collects system metrics, application logs, and OS-level data.

Analysis Speed: CloudTrail relies on Athena or external tools for slow, historical queries. CloudWatch provides real-time log streaming and fast, interactive querying.

Alerting Capabilities: CloudTrail does not have built-in alerting. CloudWatch features native, real-time alarms and metric filters.

Retention: CloudTrail automatically keeps a 90-day event history for free. CloudWatch logs require you to explicitly configure custom retention periods.

Metric value is what gets recorded when our filter spots a match in the logs. We're setting it to 1 so that each time someone accesses our secret, the counter increases by exactly

one. Default value is what gets recorded when our filter doesn't find any matches during a given time period. We're setting it to 0 so that time periods with no secret access show up as zero on our charts, rather than not showing up at all. This gives us a complete picture - we can see both when access happened AND when it didn't.

The screenshot shows the AWS CloudWatch console interface for creating a metric filter. The breadcrumb trail indicates the path: CloudWatch > Log management > mysecretsmanager-loggroup > Create metric filter. The left sidebar shows the progress: Step 3 Review and create. The main content area is titled 'Log events that match the pattern you define are recorded to the metric that you specify. You can graph the metric and set alarms to notify you.'

Filter name
GetSecretsValue

Filter pattern
GetSecretValue

☐ Enable metric filter on transformed logs
When enabled, metric filter will be applied to transformed logs. When disabled, metric filter will be applied to original logs.

Metric details

Metric namespace
Namespaces let you group similar metrics. [Learn more](#)
SecurityMetrics Create new
Namespaces can be up to 255 characters long; all characters are valid except for colon(:) at the start of the name.

Metric name
Metric name identifies this metric, and must be unique within the namespace. [Learn more](#)
Secret is accessed
Metric name can be up to 255 characters long; all characters are valid except for colon(:), asterisk(*), dollar(\$), and space().

Metric value
Metric value is the value published to the metric name when a Filter Pattern match occurs.
1
Valid metric values are: floating point number (1, 99.9, etc.), numeric field identifiers (\$1, \$2, etc.), or named field identifiers (e.g. \$requestSize for delimited filter pattern or \$.status for JSON-based filter pattern - dollar followed by alphanumeric and/or underscore (_) characters).

Default value – optional
The default value is published to the metric when the pattern does not match. If you leave this blank, no value is published when there is no match. [Learn more](#)
0

Unit – optional

Fig. 4: Metric Filter pattern

CloudWatch Alarm

The threshold setting in the CloudWatch alarm triggers an alert the moment our secret is touched.

I set:

1. The Value (Threshold = 1): I have set the threshold value to 1. This means the alarm is counting occurrences. It looks for a minimum of one single event to take action.
2. The Logic (Greater/Equal): By choosing Greater/Equal ($\geq$), we are telling CloudWatch: "If this event happens 1 time, or 5 times, or 100 times, trigger the alarm." It will ignore the number 0 (no access), but anything 1 or higher fires the alert.

3. The Combined Result: Because our metric is tracking a sensitive action (SecretIsAccessed), this threshold turns our alarm into a **Zero-Trust Security Alert**.

It means:

0 accesses = Normal state (Alarm is OK).

1+ accesses = Breach state (Alarm immediately turns to ALARM).

An SNS (Simple Notification Service) topic is a logical access point and communication channel used to group destinations and broadcast messages to multiple subscribers simultaneously.

Key Characteristics

Pub/Sub Model: It acts as a centralized "publisher" that sends a single message out to many "subscribers" instantly.

Decoupled Architecture: The system sending the alert (like your CloudWatch alarm) does not need to know who is receiving it; it just sends it to the topic.

Diverse Destinations: A single topic can deliver your alert to multiple formats at once, including Email, SMS, AWS Lambda functions, HTTPS webhooks (like Slack or Teams), and SQS queues.

How It Fits our Secret Alarm:

When our "SecretIsAccessed" threshold is breached, CloudWatch sends a notification to our SNS topic. The SNS topic then instantly routes that alert to my email.

AWS requires email confirmation because they want to make sure it's really you asking for these alerts. This helps prevent intrusion.

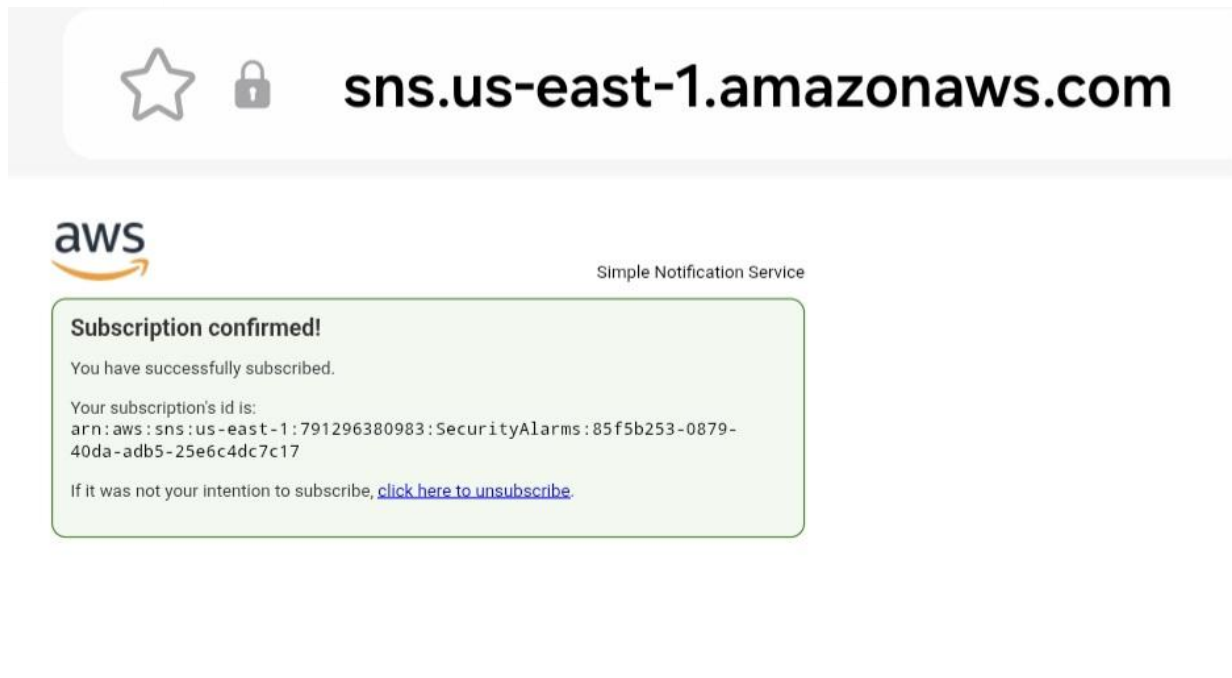


Fig. 5: Subscription confirmed AWS SNS

Troubleshooting Notification Errors

When troubleshooting the notification issues, I went through each of these consoles for troubleshooting:

CloudTrail

Log Delivery

Metric Filter

Alarm Configuration

SNS Subscription

I initially didn't receive an email before because the statistic was set to "Average". The key solution was to set it to "Sum" and 1 minute from 5 minutes.

Success!

To validate Secrets Manager and retrieved the Secret. I checked Cloudwatch if the Alarm went off. I receive the email notification when the Alarm went off.

ALARM: "Secret is accessed" in US East (N. Virginia)



AWS Notifications
To You

05:34



You are receiving this email because your Amazon CloudWatch Alarm "Secret is accessed" in the US East (N. Virginia) region has entered the ALARM state, because "Threshold Crossed: 1 out of the last 1 datapoints [2.0 (02/06/26 03:33:00)] was greater than or equal to the threshold (1.0) (minimum 1 datapoint for OK -> ALARM transition)." at "Tuesday 02 June, 2026 03:34:36 UTC".

View this alarm in the AWS Management Console:

<https://us-east-1.console.aws.amazon.com/cloudwatch/deeplink.js?region=us-east-1#alarmsV2:alarm/Secret%20is%20accessed>

Fig. 6: Alarm notification email